什么是Redis

Redis是一种基于内存的数据库，对数据的读写操作都是在内存中完成的，因此读写速度特别快，常用于缓存、消息队列、分布式锁等

为什么选择Redis作为Mysql的缓存?

高性能、高并发

Redis的数据结构有哪些?

数据结构	结构存储的值	用途
String字符串	字符串、整数、浮点数	缓存对象、常规计数、分布式锁、共享session信息
List列表	一个链表，每一个节点包含一个字符串	消息队列（存在的问题：1.生产者需要自行实现全局唯一 ID 2.不能以消费者组的形式消费数据）
Set集合	包含字符串的无序集合	聚合计算（交并差），例如点赞、共同关注、抽奖活动
Hash散列	包含键值对的无序散列表	缓存对象，购物车
Zset有序集合	也是包含键值对的散列表（Member-Score）	排序场景，例如排行榜、电话号姓名排序
BitMap		二值状态统计的场景，比如签到、判断用户登录状态
HyperLogLog		海量数据基数统计的场景
GEO		存储地理位置信息的场景
Stream		消息队列，解决用List实现消息队列存在的问题

缓存雪崩

当大量缓存数据在同一时间过期(失效)时，如果此时有大量的用户请求，都无法在 Redis 中处理，于是全部请求都直接访问数据库，从而导致数据库的压力骤增，严重的会造成数据库宕机，从而形成一系列连锁反应，造成整个系统崩溃，这就是缓存雪崩
对于缓存雪崩问题，我们可以采用两种方案解决。

1. 将缓存失效时间随机打散: 我们可以在原有的失效时间基础上增加一个随机值(比如1到 10 分钟)这样每个缓存的过期时间都不重复了，也就降低了缓存集体失效的概率。
2. 设置缓存不过期: 我们可以通过后台服务来更新缓存数据，从而避免因为缓存失效造成的缓存雪崩，也可以在一定程度上避免缓存并发问题。

缓存击穿

我们的业务通常会有几个数据会被频繁地访问，比如秒杀活动，这类被频地访问的数据被称为热点数据。如果缓存中的某个热点数据过期了，此时大量的请求访问了该热点数据，就无法从缓存中读取，直接访问数据库，数据库很容易就被高并发的请求冲垮，这就是缓存击穿的问题。

1. 互斥锁方案(Redis 中使用 setNX方法设置一个状态位，表示这是一种锁定状态)，保证同一时间只有一个业务线程重建缓存，未能获取互斥锁的请求，要么等待锁释放后重新读取缓存，要么就返回空值或者默认值。
2. 不给热点数据设置过期时间，由后台异步更新缓存，或者在热点数据准备要过期前，提前通知后台线程更新缓存以及重新设置过期时间;

缓存穿透

当发生缓存雪崩或击穿时，数据库中还是保存了应用要访问的数据，一旦缓存恢复相对应的数据，就可以减轻数据库的压力，而缓存穿透就不一样了。
当用户访问的数据，既不在缓存中，也不在数据库中，导致请求在访问缓存时，发现缓存缺失，再去访问数据库时，发现数据库中也没有要访问的数据，没办法构建缓存数据，来服务后续的请求。那么当有大量这样的请求到来时，数据库的压力骤增，这就是缓存穿透的问题。

缓存穿透的发生一般有这两种情况:
·业务误操作，缓存中的数据和数据库中的数据都被误删除了，所以导致缓存和数据库中都没有数据;
·黑客恶意攻击，故意大量访问某些读取不存在数据的业务;

应对缓存穿透的方案，常见的方案有三种。

1. 非法请求的限制:当有大量恶意请求访问不存在的数据的时候，也会发生缓存穿透，因此在 API 入口处我们要判断求请求参数是否合理，请求参数是否含有非法值、请求字段是否存在，如果判断出是恶意请求就直接返回错误，避免进一步访问缓存和数据库。
2. 设置空值或者默认值:当我们线上业务发现缓存穿透的现象时，可以针对查询的数据，在缓存中设置一个空值或者默认值，这样后续请求就可以从缓存中读取到空值或者默认值，返回给应用，而不会继续查询数据库。
3. 使用布隆过滤器快速判断数据是否存在，避免通过查询数据库来判断数据是否存在:我们可以在写入数据库数据时，使用布降过滤器做个标记，然后在用户请求到来时，业务线程确认缓存失效后，可以通过查询布隆过滤器快速判断数据是否存在，如果不存在，就不用通过查询数据库来判断数据是否存在，即使发生了缓存穿透，大量请求只会查询 Redis 和布隆过滤器，而不会查询数据库，保证了数据库能正常运行，Redis 自身也是支持布降过滤器的。

几种数据结构的内部实现

1. String类型的内部实现
   SDS（简单动态字符串）

   SDS不仅可以保存文本数据，还可以保存二进制数据

   可以O（1）时间复杂度获取字符串长度

   SDS API是安全的，拼接字符串不会造成缓冲区溢出

2. List类型的内部实现
   双向链表或压缩列表
   元素个数少于512个并且每个元素小于64字节 --> 压缩列表
   否则 --> 双向链表
   Redis3.2之后只用 --> quicklist

3. Hash类型的内部实现
   哈希表或压缩列表
   元素个数少于512个并且每个元素小于64字节 --> 压缩列表
   否则 --> 哈希表
   Redis7.0中，压缩列表数据结构已经被废弃了，交给listpack实现

4. Set类型的内部实现
   哈希表或整数集合
   元素个数少于512个 --> 整数集合
   否则 --> 哈希表

5. Zset类型的内部实现
   跳表或压缩列表
   元素个数少于512个并且每个元素小于64字节 --> 压缩列表
   否则 --> 跳表
   Redis7.0中，压缩列表数据结构已经被废弃了，交给listpack实现

Redis 是单线程吗?

Redis 单线程指的是「接收客户端请求->解析请求 ->进行数据读写等操作->发送数据给客户端」这个过程是由一个线程(主线程)来完成的，这也是我们常说 Redis 是单线程的原因。
但是，Redis 程序并不是单线程的，Redis在启动的时候，是会启动后台线程(BI0)的:
Redis 在 2.6 版本，会启动 2个后台线程，分别处理关闭文件、AOF 刷盘这两个任务;。Redis 在 4.0 版本之后，新增了一个新的后台线程，用来异步释放 Redis 内存，也就是 lazyfree 线程。例如执行 unlink key/flushdb async/flushall async 等命令，会把这些删除操作交给后台线程来执行,好处是不会导致 Redis 主线程卡顿。因此，当我们要删除一个大 key 的时候，不要使用 del 命令删除,因为 del是在主线程处理的，这样会导致 Redis 主线程卡顿，因此我们应该使用 unlink 命令来异步删除大key。